

8-Week Study Plan

Generated from the Markdown study system. Labs with executable code remain in the `Labs/` folder. This is study material, not a Cisco exam dump and not a pass guarantee.

CCNA Automation 200-901 v1.1 — 8-Week Study Plan

This plan follows the official blueprint in `200-901-CCNAAUTO_v.1.1.pdf`. It spends extra time on the two **20%** domains — **Understanding and Using APIs** and **Infrastructure and Automation** — without skipping the other four.

Use it as a calendar, not a contract. If you already write Python and Git daily, compress Weeks 1–2. If APIs or YANG are new, stretch Weeks 3–4 and 6. A faster pass is about **five weeks**; a slower pass is about **twelve**. The finish line is competency, not the date.

Daily rhythm (90–150 minutes on weekdays, longer on one weekend day):

1. Read the objective in `CCNAAUTO_COMPLETE_STUDY_GUIDE.md`.
2. Do the hands-on exercise.
3. Answer a few items in `CCNAAUTO_PRACTICE_QUESTIONS.md` for that domain.
4. Update `CCNAAUTO_Study_Tracker.xlsx` (Status, Lab Completed, Last Reviewed).

Competency language matches the tracker dropdown: Don't Know → Understand → Can Interpret → Can Perform.

Setup before Week 1: complete `CCNAAUTO_LAB_SETUP.md` through the verification checklist.

Overview

Week	Focus	Blueprint	Weight covered
1	Data formats, Python structure, TDD, process models	1.1–1.6	part of 15%
2	Git + start of REST/HTTP	1.7–1.8, 2.1–2.4	15% + start of 20%
3	APIs in depth (heaviest skill week)	2.5–2.9, start 3.1	20% core
4	Cisco platforms and construct-from-docs	3.1–3.7, 3.9	15%
5	YANG / NETCONF / RESTCONF + Docker/security	3.8, 4.1–4.8	15% + MDP
6	IaC, Ansible, Terraform, NSO, CI/CD (heaviest infra week)	4.9–4.12, 5.1–5.9	20% core
7	Interpret YANG/NETCONF/RESTCONF, diffs, diagrams; network fundamentals	5.10–5.14, 6.1–6.6	20% + 15%
8	Connectivity troubleshooting, mixed review, practice exams	6.7–6.9 + all	full blueprint

Week 1 — Software building blocks

Objectives: 1.1, 1.2, 1.3, 1.4, 1.5, 1.6

Study

- Compare XML, JSON, and YAML until you can name a snippet in one glance.
- Parse all three into Python (`dict` / `list`).
- TDD: red → green → refactor.
- Waterfall vs Agile vs Lean: sequence, feedback, waste.
- Functions, classes, modules: why they exist.
- MVC and Observer: mapping to APIs and webhooks.

Labs

- `python labs/02_data_formats/parse_formats.py`
- `python labs/01_python_basics/functions_classes_modules.py`
- `python -m unittest labs/01_python_basics/test_subnet.py` (from that folder: `python -m unittest test_subnet.py`)
- Break `interfaces.json` with a trailing comma and watch the parser fail.

Practice: Domain 1 questions 1–8 in `CCNAAUTO_PRACTICE_QUESTIONS.md`.

Expected competency: JSON/YAML/XML at **Can Interpret**. TDD and MVC at **Understand**. `unittest` at **Can Perform**.

Week 2 — Git, then REST foundations

Objectives: 1.7, 1.8.a–g, 2.1, 2.2, 2.3, 2.4

Study

- Why version control exists (history, collaboration, rollback, review).
- Perform clone, add/remove, commit, push/pull, branch, merge/conflict, diff.
- Construct a REST request from documentation (method, URL, headers, body, query).
- Webhooks vs polling.
- API constraints: rate limits, pagination, versioning, payload size, TLS.
- HTTP status families and the codes Cisco loves (200, 201, 204, 400, 401, 403, 404, 409, 415, 429, 500, 503).

Labs

- Git workflow in `CCNAAUTO_LAB_SETUP.md` section 5; read `labs/04_git/example.diff`.
- Postman: GET/POST against <https://httpbin.org>
- `python labs/03_rest_api/rest_client.py`

Practice: Domain 1 remaining Git items; Domain 2 questions on methods and status codes.

Expected competency: Git operations **Can Perform**. HTTP codes **Understand** heading toward **Can Interpret**.

Week 3 — APIs as a working skill (20% domain)

Objectives: 2.5, 2.6, 2.7, 2.8, 2.9 (review 2.1)

This is a priority week. Do not leave it “read-only.”

Study

- Troubleshoot from status + request + docs (401 vs 403 vs 404 vs 415 vs 429).
- Interpret status line, headers, body.
- Basic auth, API keys, custom/Bearer tokens.
- REST vs RPC; sync vs async.

- Python `requests: params, json=, headers, auth, timeout, response.json()`.

Labs

- `python labs/03_rest_api/troubleshoot_http.py`
- Recreate every `rest_client.py` call in Postman.
- Optional: <https://jsonplaceholder.typicode.com> if httpbin is blocked.

Practice: All remaining Domain 2 items, especially code-interpretation.

Expected competency: 2.5, 2.6, 2.7, 2.9 at **Can Perform**. 2.8 at **Understand**.

Week 4 — Cisco platforms (describe + construct)

Objectives: 3.1, 3.2, 3.3, 3.4, 3.5, 3.6, 3.7, 3.9.a–c

Study

For each platform, know: problem it solves, controller vs device, API style, auth, one representative resource.

- Network: Meraki, Catalyst Center, ACI, Catalyst SD-WAN, NSO
- Compute: UCS Manager, Intersight
- Collaboration: Webex, Webex devices, CUCM AXL vs UDS
- Security: XDR, Firepower, Secure Connect, Secure Endpoint, ISE, Secure Malware Analytics
- Device: IOS XE vs NX-OS APIs
- DevNet: Sandbox vs Code Exchange vs Learning Labs vs forums vs API docs
- Construct: list devices (3.9.a), Webex spaces/people/messages (3.9.b), list clients (3.9.c)

Labs

- `labs/05_cisco_apis/sdk_pattern.py`
- `webex_rooms.py` with a free developer token
- `meraki_list_devices.py` and `catalyst_center_devices.py` when a sandbox or API key is available
- Browse <https://devnetsandbox.cisco.com/> and <https://developer.cisco.com/codeexchange/>

If Meraki or Catalyst Center Always-On is down, still memorize the URL patterns and auth headers from the study guide and run the scripts when the tile returns.

Practice: Domain 3 platform-purpose and “which DevNet resource?” items.

Expected competency: 3.2–3.7 at **Understand**. 3.1 and 3.9 at **Can Interpret**, **Can Perform** if sandbox worked.

Week 5 — Model-driven programmability + app deployment

Objectives: 3.8, 4.1–4.8

Study

- YANG vs NETCONF vs RESTCONF, XML vs JSON, datastores, RPCs, RESTCONF URLs, ports 830 and 443.
- Edge vs private/public/hybrid cloud.
- VM vs bare metal vs containers.
- CI/CD components (source, build, test, artifact, deploy).
- Write a Python unit test.
- Interpret a Dockerfile; run an image locally.
- Secrets, encryption at rest vs in transit.

Labs

- `labs/06_yang_netconf_restconf/` against IOS XE Sandbox (launch credentials from the portal; do not reuse old blog passwords).
- `python -m unittest test_subnet.py`
- `docker build / docker run` in `labs/07_docker`

Practice: YANG/RESTCONF interpretation; Dockerfile items; unit-test items.

Expected competency: 3.8 and 5.10/5.11 preview at **Can Interpret**. Dockerfile **Can Interpret**. `docker run` **Can Perform**.

Week 6 — Infrastructure automation (20% domain)

Objectives: 4.9–4.12, 5.1–5.9

Priority week equal to Week 3.

Study

- Firewall, DNS, load balancer, reverse proxy in front of an app.
- OWASP: XSS, SQL injection, CSRF.
- Bash: `cd`, `ls`, `cp`, `mv`, `rm`, `mkdir`, `chmod`, `export`, `$PATH`.
- DevOps principles (culture + automation + measurement + sharing).
- Value of model-driven automation; controller vs device.
- CML and pyATS roles.
- CI/CD for infrastructure; IaC principles.
- Ansible vs Terraform vs NSO capabilities.

- Identify what a Python Cisco-API script is doing.
- Interpret Ansible (package, user, copy, service) and Bash workflows.

Labs

- Read and explain `labs/08_ansible/playbook.yml` out loud, then `ansible-playbook --syntax-check`.
- `terraform init/plan/apply/destroy` in `labs/09_terraform`.
- WSL Bash drills from objective 4.11.
- Optional NSO Always-On sandbox: list devices if the tile is available.

Practice: Ansible/Terraform/Bash interpretation; OWASP; IaC vs imperative scripts.

Expected competency: 5.6–5.9 at **Can Interpret**. 4.11 **Can Perform**. 4.10 and 4.12 **Understand**.

Week 7 — Interpretation skills + network fundamentals

Objectives: 5.10–5.14, 6.1–6.6

Study

- Read RESTCONF JSON and NETCONF XML replies.
- Read a basic YANG module (container, list, key, leaf, `config false`).
- Unified diff headers and hunks.
- Code review purpose.
- Sequence diagrams with API calls (sync vs async).
- MAC and VLAN; IPv4/prefix/gateway/routes.
- Switch, router, firewall, load balancer.
- Read a topology with port numbers.
- Management / control / data planes.
- DHCP, DNS, NAT, SNMP, NTP.

Labs

- Re-run RESTCONF/NETCONF gets; match output to `sample_restconf_get.json` and `sample_netconf_get-config.xml`.
- `labs/04_git/example.diff`
- Draw the mermaid topology from 6.4 in the study guide from memory.

Practice: Diffs, YANG snippets, sequence diagrams, subnetting, planes.

Expected competency: 5.10–5.12 and 5.14 at **Can Interpret**. 6.1–6.6 at **Understand**.

Week 8 — Troubleshooting, mixed review, exam readiness

Objectives: 6.7–6.9, then **all domains**

Study

- Ports: 22, 23, 80, 443, 830, plus 53, 67/68, 123, 161/162.
- Diagnose NAT, blocked port, proxy, VPN overlap.
- Network constraints on applications (latency, loss, MTU, DNS).
- `CCNAAUTO_FINAL_REVIEW.md` every day this week.
- Weak tracker rows only (Status still Don't Know or Understand on construct/interpret verbs).

Labs

- `python labs/10_network_troubleshooting/diagnose.py`
- Rebuild one lab from each folder without reading the solution first.
- Timed practice: 40–60 mixed questions, then review explanations.

Practice: Entire `CCNAAUTO_PRACTICE_QUESTIONS.md`. Then any commercial practice exam you trust — **never exam dumps**.

Expected competency

- Construct / utilize / troubleshoot / interpret objectives: **Can Perform** or at least **Can Interpret**.
- Describe / compare / explain: **Understand** or better.
- You can teach back YANG vs NETCONF vs RESTCONF, HTTP auth, and Ansible vs Terraform without notes.

Faster (5-week) and slower (12-week) variants

Five weeks: Week1 = 1.x, Week2 = 2.x, Week3 = 3.x + 3.8, Week4 = 4.x + 5.1–5.9, Week5 = 5.10–6.9 + review. Only works if you already code.

Twelve weeks: Split each week above into two, and add a full lab weekend after Weeks 3, 4, and 6. Use this if you are new to REST or Python.

Official / free study companions (in plan order)

When	Resource
All weeks	https://learningnetwork.cisco.com/s/ccnaauto-exam-topics
Weeks 2–4	https://developer.cisco.com/ and https://requests.readthedocs.io/
Weeks 4–5	https://devnetsandbox.cisco.com/
Week 5	IOS XE RESTCONF/NETCONF guides linked in the complete study guide
Week 6	https://docs.ansible.com/ and https://developer.hashicorp.com/terraform/docs
Week 7	https://developer.cisco.com/pyats/ and https://developer.cisco.com/modeling-labs/
Week 8	CCNAAUTO_FINAL_REVIEW.md

Optional paid: Official Cert Guide (Learning Matrix lists it on nearly every row) and Cisco U. CCNAAUTO path.
Not required to begin.